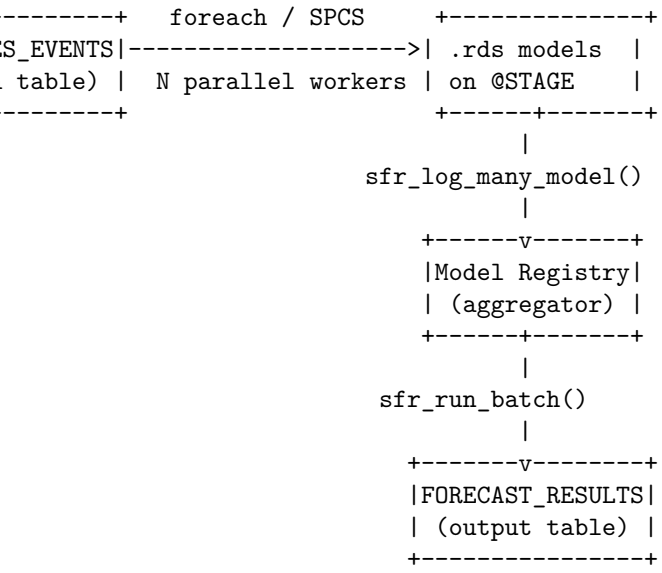
How To Build a Many-Model Forecasting Solution on Snowflake

This guide walks through the complete **many-model pattern** on Snowflake: fit hundreds or thousands of independent models in parallel using Snowpark Container Services (SPCS), persist them to a stage, register them in Model Registry as a single model version, and run server-side batch inference via `run_batch`.

The accompanying **reference notebooks** ship in snowflakeR: `inst/notebooks/workspace_parallel_spcs_setup.ipynb` and `workspace_parallel_spcs_demo.ipynb`. The code snippets below follow that pattern. Lines marked `## ADAPT` are the parts you replace for your own data and models.

1. Architecture Overview

The many-model workflow has four phases:



Two dispatch modes are available:

Mode	Mechanism	Best for
Tasks	Snowflake Tasks + SPCS jobs; fixed batch of chunks dispatched upfront	Known workload; all chunks must complete
Queue	Hybrid Table queue; dynamic feeder adds chunks as workers finish	Time-boxed runs; streaming; scale-out/in

The "screen then refine" pattern: run a fast Tasks pass (e.g. stepwise ARIMA) over all SKUs, capture accuracy metrics, then rank the worst performers and feed them into a time-boxed Queue run that applies a more expensive model contest (ARIMA + ETS + TBATS with holdout validation). The time budget is spent where it matters most.

2. Prerequisites

Snowflake objects

- A **compute pool** with enough nodes (e.g. `CPU_X64_L`, 4 nodes).
- An **image repository** with the `dosnowflake-worker` Docker image pushed.
- A **warehouse** (XS is fine for orchestration SQL).
- The role used must have `USAGE` on the compute pool and `CREATE SERVICE` / `CREATE TASK` privileges.

Local R environment

```
install.packages(c("snowflakeR", "foreach", "iterators", "forecast",
                   "ggplot2", "yaml"))
```

Authentication

Configure `~/.snowflake/connections.toml` with a named profile using key-pair (JWT) auth:

```
[my_demo_jwt]
account   = "MYORG-MYACCOUNT"
user      = "MY_USER"
authenticator = "SNOWFLAKE_JWT"
private_key_path = "~/.snowflake/keys/rsa_key.p8"
warehouse = "MY_WH"
database  = "MY_DB"
role      = "MY_ROLE"
```

3. Config and Bootstrap

The YAML config file

All environment-specific values live in a single YAML file. The reference implementation uses `snowflaker_parallel_spcs_benchmark_ak_small.yaml`:

```
## ADAPT: replace every value below with your account's objects
context:
  warehouse: MY_WAREHOUSE
  database:  MY_DATABASE
  schema:    SOURCE_DATA

parallel_lab:
  clean_room: false
  database:  MY_DATABASE
  schemas:
    source_data: SOURCE_DATA
    config:      CONFIG
    models:      MODELS
  warehouse:     MY_WAREHOUSE
  compute_pool:  MY_COMPUTE_POOL
  image_uri:     "myorg-myacct.registry.../my_db/config/images/dosnowflake-worker:latest"
  dosnowflake_stage_name: DOSNOWFLAKE_STAGE
  queue_table: DOSNOWFLAKE_QUEUE
  instance_family: CPU_X64_L

  demo_forecast_n_skus: 2000
  demo_tasks_chunks_per_job: 4
```

```
  demo_queue_n_workers: 4
  demo_queue_chunks_per_job: 8
```

Loading config and connecting

```
source("snowflakeR/inst/notebooks/parallel_spcs_workflow.R")

cfg  <- parallel_lab_load_config("path/to/my_config.yaml")
conn <- snowflakeR::sfr_connect(
  name = "my_demo_jwt",
  private_key_file = "~/.snowflake/keys/rsa_key.p8"
)
```

Bootstrap DDL

`parallel_lab_setup()` runs bootstrap SQL that creates:

- **3 schemas**: `SOURCE_DATA`, `CONFIG`, `MODELS`
- **Internal stage** `DOSNOWFLAKE_STAGE` with `DIRECTORY = (ENABLE = TRUE)`
- **Hybrid Table** `DOSNOWFLAKE_QUEUE` (for queue mode)
- **Tables**: `TRAINING_RESULTS`, `TRAINING_METRICS`, `FORECAST_RESULTS`
- **View**: `MODEL_INDEX` (parses `RELATIVE_PATH` from the stage directory table into `RUN_ID` + `PARTITION_KEY`)

```
parallel_lab_setup(conn, cfg, create_series = TRUE, n_units = 2000, n_days = 1095)
```

Set `create_series = TRUE` for synthetic data (generates 3 years of daily observations per SKU with weekly + annual seasonality, trend changepoints, and occasional outliers), or `FALSE` if your source table already exists.

ADAPT: the `SERIES_EVENTS` schema is `(UNIT_ID VARCHAR, OBS_DATE DATE, Y FLOAT)`. Replace this with your own data table's DDL.

4. The foreach Expression

The worker expression is the R code that runs **inside each SPCS container** for every unit (SKU/store/sensor/etc.). It receives an injected `unit_data` data.frame and a `unit_id` variable.

Anatomy of a worker expression

```
## This runs inside the SPCS container for each unit_id.
## `unit_data` is automatically injected by doSnowflake - it contains the rows
## from SERIES_EVENTS where UNIT_ID == unit_id.

suppressPackageStartupMessages(library(forecast))

## ADAPT: convert your data to a ts object with the right frequency
## frequency = 7 for daily data (weekly cycle); use 12 for monthly, 365.25 for
## daily with annual cycle, etc.
ts_full <- ts(unit_data$Y, frequency = 7)

## ADAPT: fit your model here (any R model that produces forecasts)
model <- auto.arima(ts_full, stepwise = TRUE)
fc    <- forecast(model, h = 30L)
acc   <- forecast::accuracy(model)
```

```r
## Contract: return a list with these fields
list(
  unit_id   = unit_id,
  model     = as.character(fc$method),
  forecast  = as.numeric(fc$mean),
  n_obs     = nrow(unit_data),
  model_obj = model,           # saved as .rds on stage
  aicc      = model$aicc,
  rmse      = acc[1, "RMSE"],
  mape      = holdout_mape,    # see holdout validation below
  mase      = acc[1, "MASE"]
)
```

Key contract: the returned list must include `unit_id` (the partition key) and `model_obj` (the fitted model object — serialized as `.rds` on the stage). Additional fields (`forecast`, metrics) are collected in the results list.

Holdout validation

For fair cross-method comparison, compute an **out-of-sample holdout MAPE** by reserving the last 30 days (or 1/4 of the series) as a test set:

```r
n <- length(ts_full)
holdout_n <- min(30L, as.integer(floor(n / 4)))

holdout_mape <- tryCatch({
  ts_train <- window(ts_full, end = time(ts_full)[n - holdout_n])
  ts_test  <- window(ts_full, start = time(ts_full)[n - holdout_n + 1])
  m_ho     <- auto.arima(ts_train, stepwise = TRUE)
  fc_ho    <- forecast(m_ho, h = holdout_n)
  mean(abs((ts_test - fc_ho$mean) / ts_test)) * 100
}, error = function(e) NA_real_)
```

5. Tasks Mode (Fixed Batch)

Tasks mode dispatches all chunks upfront using a Snowflake Task DAG. Each task launches an SPCS job service that processes one chunk of `unit_ids`.

```r
tasks_out <- parallel_lab_run_tasks_demo(conn, cfg,
  run            = TRUE,
  arima_stepwise = TRUE,
  forecast_h     = 30L,
  save_models    = TRUE
)
```

How it works

1. `registerDoSnowflake(conn, mode = "tasks", ...)` sets up the backend.
2. The `foreach` loop iterates over `unit_ids`; doSnowflake chunks them into `chunks_per_job` groups.
3. Each chunk becomes an SPCS job service that:
   - Bulk-reads its partition of `SERIES_EVENTS` via the `data_query` spec
   - Runs `mclapply` over the units within the container
   - Writes `.rds` model files to `@STAGE/models/<run_id>/<unit_id>.rds`
   - Writes result `.rds` back to `@STAGE/<job>/results/result_<chunk>.rds`
4. The orchestrator polls until all chunks complete, then collects results.

Chunk sizing

Balance parallelism vs overhead:

- **Too few chunks** (e.g. 1): all work in one container, no parallelism across nodes.
- **Too many chunks** (e.g. 2000, one per SKU): excessive job-launch overhead.
- **Sweet spot**: 4–20 chunks for 2000 units. Each container gets 100–500 SKUs and uses `mclapply` for intra-node parallelism.

6. Queue Mode (Time-Boxed Streaming)

Queue mode uses a Hybrid Table as a work queue. A **dynamic feeder** inserts rows as workers pull them, up to a time budget.

```
queue_out <- parallel_lab_run_queue_timeboxed(conn, cfg,
  run            = TRUE,
  time_limit_sec = tasks_out$elapsed,   # match Tasks wall time
  model_contest  = TRUE,                # ARIMA + ETS + TBATS contest
  forecast_h     = tasks_out$params$forecast_h,
  n_chunks       = 40L,
  unit_ids       = worst_skus           # worst-first from Tasks
)
```

Dynamic feeder pattern

Unlike Tasks, the Queue does **not** dispatch all work upfront. The feeder loop:

1. Checks how many chunks are `PENDING` in the queue.
2. If slots are available and the time budget hasn't expired, inserts the next chunk.
3. Launches SPCS workers that claim rows via compare-and-swap (CAS): `UPDATE ... SET STATUS = 'RUNNING' WHERE STATUS = 'PENDING' LIMIT 1`.
4. When the feeder's time budget expires, it stops feeding but waits for in-flight chunks to finish.

This gives a natural "time-boxed" behavior: you spend exactly as much wall time as you budget, processing as many chunks as workers can handle.

Accuracy-ranked feeding

By ranking SKUs worst-first (highest holdout MAPE from the Tasks run), the time budget is spent where it has the greatest impact:

```
worst_skus <- parallel_lab_rank_skus_by_accuracy(
  run_results = tasks_out,
  metric      = "mape",
  descending  = TRUE
)
```

Model contest

When `model_contest = TRUE`, each SKU's worker expression runs a holdout- validated competition:

1. Split the daily series into training (~1065 days) and test (30 days).
2. Fit three candidates:
   - `auto.arima(stepwise = FALSE)` (exhaustive ARIMA search)
   - `ets()` (exponential smoothing)
   - `tbats()` (Trigonometric seasonality, Box-Cox, ARMA errors, Trend, Seasonal — handles multiple seasonal periods natively, e.g. weekly + annual cycles in daily data)
3. The candidate with the lowest test-set MAPE wins.

4. The winner is refit on the full series for the production model.

TBATS is particularly well-suited to this contest because the daily synthetic data has **two nested seasonal cycles** (weekly period = 7, annual period = 365.25) that ARIMA and ETS cannot capture simultaneously.

ADAPT: replace the three candidate model families with whatever makes sense for your domain (Prophet, XGBoost, LSTM, etc.).

7. Metrics and Comparison

Persisting metrics

After each run, save per-unit accuracy metrics to the `TRAINING_METRICS` table:

```
parallel_lab_save_metrics(conn, cfg, tasks_out)
parallel_lab_save_metrics(conn, cfg, queue_out)
```

This writes `(MODEL_RUN_ID, UNIT_ID, ARIMA_MODEL, AICC, RMSE, MAPE, MASE, N_OBS)` for every successfully fitted unit. The `MAPE` column stores the **holdout** (out-of-sample) MAPE — not the training MAPE.

Why holdout MAPE?

Training MAPE reflects how well the model fits historical data. Holdout MAPE reflects how well it **predicts unseen data**. When comparing a simple stepwise ARIMA (Tasks) against a model contest (Queue), training MAPE will look similar because both methods fit the full series. The difference only shows up in out-of-sample performance.

Plotting

Single-run histogram:

```
parallel_lab_plot_accuracy(tasks_out, metric = "mape")
```

Faceted comparison (Tasks vs Queue, matched SKUs only):

```
parallel_lab_plot_accuracy_comparison(
  tasks_out, queue_out,
  metric = "mape",
  matched_only = TRUE
)
```

The `matched_only = TRUE` parameter restricts the comparison to SKUs that appear in both runs, making the distributions directly comparable.

8. Model Registry

Registering N models as one version

After training, each SKU's `.rds` model file lives at `@STAGE/models/<run_id>/<unit_id>.rds`. The stage's `DIRECTORY` table tracks all files. The `MODEL_INDEX` view extracts `RUN_ID` and `PARTITION_KEY` from the file paths.

`sfr_log_many_model()` registers a **CustomModel aggregator** in Model Registry. The aggregator:

- Carries a `model_index.json` mapping `UNIT_ID` → stage path
- Exposes `@partitioned_api predict()` for `run_batch` (TABLE_FUNCTION)
- Exposes `@inference_api predict_single()` for SPCS service (FUNCTION)
- Registers in O(1) time (~8–13s regardless of partition count)

```
reg_tasks <- parallel_lab_register_models(conn, cfg,
  model_run_id = tasks_out$model_run_id,
  forecast_h   = tasks_out$params$forecast_h
)
```

The Ray shuffle bug (SNOW-2193288)

Model Registry's `run_batch` uses Ray internally for partitioned dispatch. When the number of unique partition keys is much less than ~200, Ray's shuffle mechanism can fail. **Always ensure >= 200 partition keys** when using `run_batch`. The reference implementation warns if this threshold is not met.

9. Batch Inference

Server-side partitioned inference

`sfr_run_batch()` launches an SPCS batch job that:

1. Partitions the input DataFrame by `UNIT_ID`.
2. For each partition, the `RManyModelAggregator` downloads the `.rds` model from stage, loads it via `rpy2`, and runs `forecast::forecast(model, h=...)`.
3. Returns a DataFrame of `(PARTITION_KEY, PERIOD, FORECAST, LOWER_80, UPPER_80, LOWER_95, UPPER_95, STATUS)`.

```
fc_df <- parallel_lab_run_inference(conn, cfg,
  model_run_id = tasks_out$model_run_id,
  forecast_h   = tasks_out$params$forecast_h
)
```

The results are written to the `FORECAST_RESULTS` table with columns: `RUN_ID, UNIT_ID, HORIZON, FORECAST_DATE, POINT_FORECAST, LO_80, HI_80, LO_95, HI_95, STATUS`.

Input contract

The input DataFrame needs at minimum a `UNIT_ID` column (the partition key). Additional columns are ignored by the aggregator — it loads the model from stage rather than re-training.

10. Monitoring

The reference implementation includes two Streamlit dashboards:

- **`streamlit_benchmark_monitor.py`**: live progress for Tasks vs Queue benchmark runs, filtered by `JOB_ID`.
- **`streamlit_parallel_demo_monitor.py`**: general-purpose SPCS service and queue monitoring.

Deploy to Snowflake (Streamlit-in-Snowflake) with:

```
python deploy_streamlit_monitor.py \
  --connection-name my_demo_jwt \
  --private-key-file ~/.snowflake/keys/rsa_key.p8
```

Tip: scope dashboard queries by `JOB_ID` to avoid showing cumulative counts from previous runs.

11. Lessons Learned

JWT expiry and stale sessions

`sfr_connect()` auto-detects Workspace Notebook sessions. When running in RStudio with explicit `name` / `account` parameters, it skips auto-detection to avoid wrapping a stale, expired Python session. If you see `JWT token is invalid` after a timeout, explicitly reconnect:

```r
conn <- snowflakeR::sfr_connect(
  name = "my_demo_jwt",
  private_key_file = "~/.snowflake/keys/rsa_key.p8"
)
dbi_con <- snowflakeR::sfr_dbi_connection(conn)
```

RStudio Connections Pane

Call `sfr_dbi_connection(conn)` after `sfr_connect()` to populate the Connections Pane with database catalog info. If schemas or tables don't appear, disconnect in the pane and reconnect — RStudio caches connection observer closures.

Setting the right ts frequency

Use `ts(data, frequency = 7)` for daily data with a weekly cycle, or `frequency = 12` for monthly data. Without the correct frequency, `auto.arima` cannot learn seasonal patterns and forecasts will be flat. For daily data with both weekly and annual cycles, TBATS auto-detects multiple seasonal periods from the `frequency` attribute.

Chunk count parity

When comparing Tasks vs Queue, use chunk counts that give comparable total work. The Queue's dynamic feeder means fewer chunks may complete within the time budget — that's by design (time-boxed, not work-boxed).

`bquote` vs `quote` for parameter substitution

The worker expression uses `bquote()` with `.()` splicing to inject parameter values (e.g. `.(forecast_h)`, `.(arima_stepwise)`) at definition time. Using `quote()` would capture the variable *names*, not their *values*, causing failures when the expression runs in a remote container where those variables don't exist.

```r
forecast_expr <- bquote({
  model <- auto.arima(ts_full, stepwise = .(arima_stepwise))
  fc    <- forecast(model, h = .(forecast_h))
  # ...
})
```

Further Reading

- `vignette("parallel-dosnowflake")`: the low-level `registerDoSnowflake` API reference (foreach backend, generic SPCS executor, crew/mirai workers).
- **Reference notebooks**: `inst/notebooks/workspace_parallel_spcs_setup.ipynb`, `workspace_parallel_spcs_dem` contains the complete runnable demo with configs, deploy scripts, and tests.
- **Function reference**: `?sfr_log_many_model`, `?sfr_run_batch`, `?sfr_model_registry`, `?registerDoSnowflake`.